CHAPTER 5

Addressing Modes

Immediate · direct · indirect · register · register indirect · indexed · base · PC-relative · auto-increment

QUICK TRICK

EA = effective address of the operand. Immediate: operand is *in* the instruction. Direct: EA = A. Indirect: EA = M[A]. Indexed: EA = A + R. PC-relative: **EA = PC (already incremented) + offset**. Count the memory references.

5.1 Why many modes?

● LEVEL 1 · BEGINNER

The problem. The same instruction must reach constants, variables, array elements, pointers and code. **The idea.** The instruction names *how* to find its operand. **Analogy.** Finding a book: "this one in my hand" (immediate), "shelf 5" (direct), "the slip says which shelf" (indirect).

5.2 The modes

● LEVEL 2 · EXAM

Mode	Effective address / operand	Operand memory refs	Typical use
Implied	operand fixed (e.g. accumulator, stack top)	0	CLC, PUSH
Immediate	operand = value in instruction	0	Constants
Register	operand = R	0	Fast variables
Direct (absolute)	EA = A	1	Static variables
Indirect	EA = M[A]	2	Pointers
Register indirect	EA = (R)	1	Pointers, arrays
Indexed	EA = A + (index R)	1	Array elements
Base register	EA = (base R) + displacement	1	Relocation, records
PC-relative	EA = PC + offset	1 (for data)	Branches, position-independent code
Auto-increment	EA = (R); then R ← R + d	1	Walking through arrays
Auto-decrement	R ← R − d; then EA = (R)	1	Stacks

5.3 Worked example

● LEVEL 2 · EXAM

Memory: M[100] = 200, M[200] = 300. Registers: R1 = 100. Instruction field A = 100. Value loaded:

Mode	Value fetched	Reasoning
Immediate #100	**100**	the constant itself
Direct 100	**200**	M[100]
Indirect (100)	**300**	M[M[100]] = M[200]
Register R1	**100**	contents of R1
Register indirect (R1)	**200**	M[R1] = M[100]
Indexed 100(R1)	**300**	EA = 100 + 100 = 200, M[200]

5.4 PC-relative branches

● LEVEL 3 · GATE

When the instruction is executed the PC has **already been incremented** to the next instruction. **Example.** Branch at address 3000, instruction length 4 bytes, offset +12: target = 3000 + 4 + 12 = **3016**.

- **Auto-increment:** MOV (R1)+, R2 with R1 = 1000 and 4-byte words loads M[1000] into R2, then R1 becomes **1004**.
- **Total memory references** of an instruction = 1 (fetch, per word) + operand references. ADD R1, @100 (one-word instruction, indirect) = 1 + 2 = **3**.
- Base register mode lets a program be **relocated** by changing only the base register. Indexed mode keeps the array start in the instruction and the index in a register.

5.5 Common mistakes

COMMON MISTAKE

- Using the old PC (instruction address) instead of the incremented PC for PC-relative.
- Forgetting the extra memory access in indirect mode.
- Confusing indexed (address field + register) with register indirect (register only).
- Counting the instruction fetch as zero memory references.

QUICK TRICK

Memory line: "Immediate: here. Direct: there. Indirect: ask there where."

GATE QUICK CHECK — Chapter 5

```
Imm 0 | Reg 0 | Direct 1 | Reg-indirect 1 | Indexed 1 | Indirect 2   (operand memory refs)
Indexed EA=A+R   Base EA=BR+disp   PC-rel EA=PC_next+offset   Auto-inc EA=(R), R+=d
arrays -> indexed   pointers -> indirect   relocation -> base   branches -> PC-relative
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Which modes need no memory access for the operand?

Q2. How many memory references fetch an operand in indirect mode?

Q3. Which mode suits array access? Which suits position-independent branches?

Q4. Indexed mode: A = 500, index register = 20. EA?

● LEVEL 2 · EXAM

Q5. Using the example in 5.3, what does Indirect (100) fetch?

Q6. Branch at 3000, 4-byte instruction, offset +12 (PC-relative). Target?

Q7. MOV (R1)+, R2 with R1 = 1000, 4-byte words. Final R1?

Q8. Total memory references for ADD R1, @100 as a one-word instruction?

● LEVEL 3 · GATE

Q9. A 16-bit instruction uses base register mode with an 8-bit unsigned displacement. Range of EA with base register = 4000?

Q10. Why is PC-relative addressing good for position-independent code?

Q11. Instruction at 2000 occupies 2 bytes; PC-relative offset is −6. Target?

Answers

Q	Solution
1	**Immediate** and **register** (and implied).
2	**2**.
3	**Indexed**; **PC-relative**.
4	**520**.
5	**300**.
6	3000 + 4 + 12 = **3016**.
7	**1004**.
8	1 fetch + 2 (indirect) = **3**.
9	EA from **4000 to 4255**.
10	The target is expressed as a distance from the current instruction, so it stays correct wherever the program is loaded.
11	2000 + 2 − 6 = **1996**.

Next: Chapter 6 — ALU Design.

AMJESH
singhamjesh.github.io